

Pourquoi chaque champ & comment le collecter avec Ansible (multi-OS)

Pour chaque champ du schéma agnostique : pourquoi on en a besoin, et comment l'obtenir via Ansible sur **AIX** · **Linux** · **Windows (tous de plein droit)**.

Approche Ansible multi-OS

Le principe directeur est qu'un **seul rôle Ansible** alimente le document JSON canonique (un par hôte), mais que **la commande de collecte se ramifie par plateforme**. On exploite les groupes d'inventaire (`aix`, `linux_rhel`, `linux_debian`, `linux_suse`, `windows`) et la variable `ansible_facts['os_family']` / `ansible_facts['system']` pour router vers le bon jeu de commandes. Le résultat de chaque branche est normalisé vers le **dénominateur universel** `filesystems[]` plus la `storage_topology` neutre (`groups` / `volumes` / `disks` + `*_kind`).

gather_facts par famille et connectivité mixte dans le même play

- **Linux / AIX** : connexion **SSH** (`ansible_connection: ssh`), `gather_facts: yes` pour récupérer `os_family`, `distribution`, `architecture`, `processor_architecture`. Ces facts servent uniquement à l'**identité** et au **routing** ; ils ne servent **PAS** à la mesure de capacité.
- **Windows** : connexion **WinRM** (`ansible_connection: winrm`, `ansible_winrm_transport: kerberos|ntlm`, port 5985/5986), `gather_facts: yes` via `ansible.windows` pour `ansible_os_name`, `ansible_architecture`. La collecte de stockage se fait via `ansible.windows.win_powershell`.
- **Même play, groupes différents** : on peut piloter SSH et WinRM dans le **même play** en ciblant `hosts: all` et en laissant chaque groupe d'inventaire porter ses variables de connexion (`group_vars/windows.yml` → `ansible_connection: winrm`, `group_vars/linux.yml` → `ansible_connection: ssh`). Les tâches sont gardées par `when: ansible_facts['os_family'] == ...` ou `when: inventory_hostname in groups['windows']`.

Pourquoi éviter `ansible_lvm` / `size_g`

- `ansible_facts['lvm']` n'existe **pas sur AIX ni Windows** et dépend d'un fact custom non garanti ; il n'expose pas les VG free/extents de façon homogène.
- Les champs `*_g` / `size_total` formatés en Go arrondissent et perdent la précision. **Le schéma exige TOUJOURS des octets**. On force donc `df -B1` (Linux), `df -k` puis multiplication $\times 1024$ (AIX), `vgs/lvs/pvs --units b --nosuffix`, et sous Windows les API renvoient déjà des octets (`Size`, `SizeRemaining`).

Commandes brutes ramifiées par plateforme (lecture seule)

Toutes les commandes sont **strictement en lecture seule** ; aucune n'écrit, ne monte, ni ne modifie le stockage. On les lance via `ansible.builtin.command` (jamais `shell` sauf pipe nécessaire) avec `changed_when: false`.

Plateforme	Topologie / groupes	Filesystems	Paging
AIX	<code>lsvg</code> , <code>lsvg <vg></code> , <code>lsvg -l <vg></code> , <code>lspv</code> , <code>lslv <lv></code>	<code>df -k</code> , <code>df -v</code> (inodes JFS2), <code>lsfs</code>	<code>lspcs -a</code>
RHEL / Oracle / Rocky / Alma	<code>vgs --units b --nosuffix</code> , <code>lvs --units b --nosuffix</code> , <code>pvs --units b --nosuffix</code>	<code>df -B1</code> , <code>df -i</code> , <code>xfs_info <mp></code>	<code>swapon --show --bytes</code>
Debian / Ubuntu	<code>vgs/lvs/pvs --units b</code> (si LVM)	<code>df -B1</code> , <code>df -i</code> , <code>tune2fs -l <dev></code> (reserved blocks ext)	<code>swapon --show --bytes</code>
SUSE / SLES	<code>vgs/lvs/pvs</code> (si LVM)	<code>btrfs filesystem usage -b <mp></code> , <code>btrfs qgroup show -b <mp></code> , <code>df -B1</code> (fallback non-btrfs)	<code>swapon --show --bytes</code>
Windows	<code>Get-StoragePool</code> , <code>Get-VirtualDisk</code> , <code>Get-Disk</code> , <code>Get-PhysicalDisk</code>	<code>Get-Volume</code> , <code>Get-Partition</code>	<code>Get-CimInstance Win32_PageFileUsage</code>

SUSE / BTRFS — point critique : `df` est **trompeur** sur btrfs (sous-volumes + snapshots partagent l'espace). On utilise `btrfs filesystem usage -b` (octets exacts : `Device size` , `Used` , `Free (estimated)`) et `btrfs qgroup show -b` pour les vrais octets par sous-volume.

Windows première classe — via `ansible.windows.win_powershell`

La collecte Windows est livrée **maintenant, à la même profondeur** qu'AIX/Linux (pas de « phase 2 »). On exécute un bloc PowerShell unique qui appelle `Get-Volume` , `Get-Partition` , `Get-Disk` , `Get-PhysicalDisk` , `Get-StoragePool` , `Get-VirtualDisk` , `Get-CimInstance Win32_PageFileUsage` et `Get-Acl` , puis **renvoie un objet JSON** via `ConvertTo-Json -Depth 6` . Correspondance vers le schéma neutre :

- `Get-StoragePool` → `groups[]` avec `group_kind: storage-pool` (capabilities.resiliency).
- `Get-VirtualDisk` → `volumes[]` avec `volume_kind: virtual-disk` (`redundancy = ResiliencySettingName`).
- Disque basique** (sans Storage Spaces) → partitions exposées comme `volumes[]` (`volume_kind: partition`), `groups[]` **vide**.
- `Get-Volume` → `filesystems[]` (`mount_point` = lettre `C:` ou point de montage dossier, `fstype` = `ntfs` / `refs` , `access_model: ntfs-acl` , `inodes_* = null` , `reserved_bytes = null` , `mount_options = null`).
- `Get-Acl` → `acl_summary` **non sensible uniquement** (nombre d'ACE, héritage activé, présence de groupes intégrés type `BUILTIN\Administrators` , `SYSTEM`) — **jamais** les SID utilisateurs nominatifs ni les chemins sensibles.
- Volume système** = `BootDevice` /là où Windows est installé → `storage_class` dérivé `system` ; les autres → `data` (généralise `rootvg` / `datavg` à Windows : `C:` = système).

Assemblage du JSON agnostique, planification et garde-fous

- Assemblage** : chaque branche enregistre sa sortie (`register:`), on parse (`from_json` , ou parsing ligne à ligne pour `lsvg` / `df`), puis on construit le document via `set_fact` en posant `filesystems` , `storage_topology` , `paging` et les métadonnées d'identité. Le lien FS ↔ topologie se fait par **clés**

`group_ref` / `backing_ref` (pas d'imbrication), ce qui permet au pipeline de **dériver** `storage_class` (système vs data).

- **Planification** : orchestration via **AWX / Ansible Tower** (Job Template + Schedule, p. ex. quotidien décalé), inventaire dynamique pour la flotte ≥ 500 hôtes, `forks` ajustés et `serial` pour lisser la charge.
- **Sécurité lecture seule** : `changed_when: false`, `check_mode` compatible, aucun module modifiant l'état ; les commandes d'agrégation de contenu sont **bornées**.
- **Garde-fous d'agrégation** (2e vague, `content_aggregates`) : sous Unix `du --max-depth=1 -b` et `find avec limites` (`-maxdepth`, `timeout`, exclusion des FS réseau/pseudo via `-xdev`) ; sous Windows `Get-ChildItem` avec `-Depth 1` et `Measure-Object -Sum Length`. **Métadonnées et agrégats non sensibles uniquement** (tailles, comptes, âges) — jamais de contenu de fichier, jamais de noms sensibles ; **résumé ACL** seulement côté Windows.

Pourquoi chaque champ + comment l'obtenir

Identité et métadonnées de collecte (universel, tout OS)

Champ	Pourquoi	Comment (par plateforme)
<code>host_id</code> , <code>hostname</code> , <code>gsma_code</code>	Clé de jointure flotte, corrélation CMDB, objectif 4 (efficacité opérationnelle)	<code>ansible_facts['fqdn']</code> / <code>inventory_hostname</code> (tous) ; <code>gsma_code</code> depuis <code>host_vars</code> / inventaire
<code>host_role_env</code>	Calibrer les standards (prod vs hors-prod), objectif 1	Variable d'inventaire (tous OS)
<code>platform</code> (<code>aix</code> / <code>linux</code> / <code>windows</code>)	Discriminateur de branche de collecte	<code>ansible_facts['system']</code> (Unix) / <code>ansible_os_name</code> (Windows)
<code>os_family</code> , <code>os_distribution</code> , <code>os_version</code>	Choix du standard par distro (XFS vs ext4 vs btrfs vs jfs2 vs ntfs)	<code>ansible_facts['os_family']</code>
<code>architecture</code>	Présence conditionnelle (<code>/boot/efi</code> UEFI x86_64), compatibilité	<code>ansible_facts['architecture']</code> (Unix) / <code>ansible_architecture</code> (Windows)
<code>snapshot_id</code> , <code>scan_timestamp</code>	Séries temporelles, prévision de croissance, objectif 3	<code>ansible_date_time.iso8601</code> + UUID généré au run
<code>collection_method</code> , <code>scan_confidence</code>	Traçabilité, qualité de la donnée	Constante du rôle (<code>ansible-ssh</code> / <code>ansible-winrm</code>) ; confiance baissée si fallback (<code>df</code> sur btrfs)

filesystems[] — dénominateur universel

Champ	Pourquoi	Comment (par plateforme)
<code>mount_point</code>	Unité de déviation FS (objectif 1) ; détecter données mal placées (objectif 5)	AIX <code>df -k</code> ; Linux <code>df -B1</code> ; Windows <code>Get-Volume</code> (DriveLetter <code>C:</code> ou <code>Get-Partition</code> <code>AccessPaths</code> pour montage dossier)
<code>label</code> , <code>fstype</code>	Le type pilote la méthode de mesure et le standard	<code>df -v / lsfs</code> (AIX jfs2) ; <code>df -B1</code> colonne FS / <code>lsblk -f</code> (Linux) ; Windows <code>Get-Volume</code> (<code>FileSystemType</code> ntfs/refs)

Champ	Pourquoi	Comment (par plateforme)
<code>size_total_bytes</code> , <code>size_used_bytes</code> , <code>size_available_bytes</code>	Cœur capacité, prévention FS pleins (objectif 3), TOUJOURS octets	AIX <code>df -k ×1024</code> ; Linux <code>df -B1</code> ; SUSE <code>btrfs filesystem usage -b</code> ; Windows <code>Get-Volume (Size , SizeRemaining → used = Size-SizeRemaining)</code>
<code>reserved_bytes</code> (nullable)	Espace réservé root fausse le « plein réel »	ext seulement <code>tune2fs -l</code> (Block count×res%×blocksize) ; null sur xfs/jfs2/ntfs/btrfs
<code>inodes_total</code> , <code>inodes_used</code> (nullable)	Saturation d'inodes = FS « plein » sans octets pleins (objectif 3)	AIX <code>df -v</code> ; Linux <code>df -i</code> ; null sur Windows (NTFS/ReFS)
<code>access_model</code>	Modèle de droits selon OS	<code>posix-mount-options</code> (Unix) / <code>ntfs-acl</code> (Windows)
<code>mount_options</code>	Conformité (noexec/nosuid), risque (objectif 5) ; null Windows	<code>/proc/mounts</code> / <code>mount</code> / <code>df</code> (Unix) ; null (Windows)
<code>acl_summary</code>	Résumé droits Windows non sensible (objectif 5) ; null Unix	Windows <code>Get-Acl</code> résumé (nb ACE, héritage, principaux intégrés) ; null (Unix)
<code>backing_kind</code> , <code>backing_ref</code> , <code>group_ref</code>	Lien FS ↔ topologie par clé → dérive <code>storage_class</code> (système/data)	lvm-lv/partition (Linux) ; lvm-lv (AIX) ; virtual-disk/partition/storage-space (Windows)
<code>content_aggregates</code>	Optimisation : données obsolètes/inutiles/mal placées (objectifs 2 & 5)	<code>du --max-depth=1</code> / <code>find</code> borné (Unix) ; <code>Get-ChildItem</code> agrégé (Windows)

storage_topology — groups / volumes / disks (neutre + *_kind)

Champ	Pourquoi	Comment (par plateforme)
<code>groups[].group_name</code> , <code>group_kind</code>	VG/pool = unité de croissance et de déviation groupe (objectif 1)	AIX <code>lsvg (lvm-vg)</code> ; Linux <code>vgs (lvm-vg)</code> ; Windows <code>Get-StoragePool (storage-pool)</code> ; vide pour partition simple/basic disk
<code>groups[].size_total_bytes</code> , <code>size_free_bytes</code> , <code>member_disk_count</code> , <code>state</code> , <code>extent_size_bytes</code>	Capacité d'extension, marge avant saturation (objectif 3)	AIX <code>lsvg <vg></code> (PP size×total/free PPs) ; Linux <code>vgs --units b</code> (VSize/VFree/Ext) ; Windows <code>Get-StoragePool (Size , AllocatedSize)</code>
<code>groups[].capabilities</code>	Limites structurelles (nullable AIX max_lvs/max_pps/vg_type ; Windows resiliency)	AIX <code>lsvg <vg></code> ; Windows <code>Get-StoragePool / Get-VirtualDisk ResiliencySettingName</code>
<code>volumes[].volume_name</code> , <code>volume_kind</code> , <code>group_ref</code> , <code>size_bytes</code> , <code>type</code> , <code>redundancy</code>	LV/partition/vdisk = porteur du FS, redondance = fiabilité (objectif 3)	AIX <code>lslv (lvm-lv)</code> ; Linux <code>lvs --units b (lvm-lv)/partition</code> ; Windows <code>Get-VirtualDisk (virtual-disk)</code> ou <code>Get-Partition (partition)</code>
<code>disks[].disk_name</code> , <code>size_bytes</code> , <code>free_bytes</code> , <code>group_ref</code> , <code>media_type</code>	Capacité physique, type média (SSD/HDD) pour optimisation (objectif 2)	AIX <code>lspv</code> ; Linux <code>pvs --units b / lsblk</code> ; Windows <code>Get-Disk / Get-PhysicalDisk (MediaType)</code>

paging[]

Champ	Pourquoi	Comment (par plateforme)
name , kind	Le swap/pagefile sur disque système = risque & capacité (objectifs 3 & 5)	aix-paging-lv / linux-swap-partition / linux-swap-file / windows-pagefile
size_total_bytes , size_used_bytes	Dimensionnement, saturation mémoire virtuelle	AIX <code>lspcs -a</code> (MB→×1048576) ; Linux <code>swapon --show=NAME,SIZE,USED --bytes</code> ; Windows <code>Get-CimInstance Win32_PageFileUsage</code> (<code>AllocatedBaseSize</code> / <code>CurrentUsage</code> en Mo →×1048576)

Exemples de tâches Ansible

1. Filesystems en octets — Linux (df -B1 + df -i)

```

- name: "Linux | df octets + inodes (lecture seule)"
  when: ansible_facts['system'] == 'Linux'
  block:
    - name: "df -B1 (octets exacts)"
      ansible.builtin.command:
        cmd: df -B1 -P --output=source,target,fstype,size,used,avail
      register: df_bytes
      changed_when: false

    - name: "df -i (inodes)"
      ansible.builtin.command:
        cmd: df -i -P --output=target,itotal,iused
      register: df_inodes
      changed_when: false

    - name: "Construire filesystems[] (parsing des lignes)"
      ansible.builtin.set_fact:
        fs_linux: >-
          [{ df_bytes.stdout_lines[1:] | map('split') | map('community.general.dict_kv')
            | default([]) }]
      # En pratique : boucle/filtre custom pour mapper source/target/fstype/size/used/avail
      # vers mount_point/fstype/size_total_bytes/size_used_bytes/size_available_bytes.
```

2. SUSE / BTRFS — vrais octets (df est trompeur)

```

- name: "SUSE | btrfs filesystem usage en octets"
  when: ansible_facts['os_family'] == 'Suse'
  block:
```

```

- name: "Lister les points de montage btrfs"
  ansible.builtin.shell:
    cmd: "findmnt -nro TARGET -t btrfs"
  register: btrfs_mounts
  changed_when: false

- name: "btrfs filesystem usage -b par montage"
  ansible.builtin.command:
    cmd: "btrfs filesystem usage -b {{ item }}"
  loop: "{{ btrfs_mounts.stdout_lines }}"
  register: btrfs_usage
  changed_when: false

- name: "qgroups (octets par sous-volume)"
  ansible.builtin.command:
    cmd: "btrfs qgroup show -b --raw {{ item }}"
  loop: "{{ btrfs_mounts.stdout_lines }}"
  register: btrfs_qgroups
  changed_when: false
  failed_when: false # qgroups peut être désactivé : confiance abaissée

```

3. AIX — parsing lsvg vers groups[]

```

- name: "AIX | topologie LVM (rootvg/datavg)"
  when: ansible_facts['system'] == 'AIX'
  block:
    - name: "Lister les VG"
      ansible.builtin.command:
        cmd: lsvg
      register: aix_vgs
      changed_when: false

    - name: "Détail de chaque VG (octets via PP size x PPs)"
      ansible.builtin.command:
        cmd: "lsvg {{ item }}"
      loop: "{{ aix_vgs.stdout_lines }}"
      register: aix_vg_detail
      changed_when: false

    - name: "df -k et df -v (octets x1024 + inodes jfs2)"
      ansible.builtin.command:
        cmd: "{{ item }}"
      loop:
        - df -k
        - df -v

```

```

register: aix_df
changed_when: false

# PP SIZE (Mo) x TOTAL PPs x 1048576 => size_total_bytes ; FREE PPs => size_free_bytes
# group_kind: lvm-vg ; capabilities.max_lvs/max_pps/vg_type depuis la sortie lsvg.

```

4. Windows — Get-Volume / Get-Disk / topologie via win_powershell → ConvertTo-Json

```

- name: "Windows | stockage complet via PowerShell -> JSON"
  when: inventory_hostname in groups['windows']
  ansible.windows.win_powershell:
    script: |
      $ErrorActionPreference = 'Stop'
      $boot = (Get-CimInstance Win32_OperatingSystem).SystemDrive # ex 'C:'

      $volumes = Get-Volume | Where-Object DriveType -eq 'Fixed' | ForEach-Object {
        [pscustomobject]@{
          mount_point      = if ($_.DriveLetter) { "$($_.DriveLetter):" } else { ($_.Path) }
          label            = $_.FileSystemLabel
          fstype            = $_.FileSystemType.ToString().ToLower()
          size_total_bytes  = [int64]$_ .Size
          size_available_bytes = [int64]$_ .SizeRemaining
          size_used_bytes   = [int64]($_.Size - $_.SizeRemaining)
          access_model      = 'ntfs-acl'
          storage_class     = if ("${($_.DriveLetter):}" -eq $boot) { 'system' } else { 'data' }
        }
      }

      $disks = Get-Disk | ForEach-Object {
        [pscustomobject]@{
          disk_name = $_.FriendlyName
          size_bytes = [int64]$_ .Size
          media_type = (Get-PhysicalDisk -ErrorAction SilentlyContinue |
                        Where-Object DeviceId -eq $_.Number).MediaType
        }
      }

      $pools = @(Get-StoragePool -IsPrimordial $false -ErrorAction SilentlyContinue | ForEach-Object {
        [pscustomobject]@{
          group_name      = $_.FriendlyName
          group_kind      = 'storage-pool'
          size_total_bytes = [int64]$_ .Size
          size_free_bytes  = [int64]($_.Size - $_.AllocatedSize)
        }
      })

```

```

        capabilities      = @{ resiliency = $_.ResiliencySettingNameDefault }
    }
})
$vdisk = @(Get-VirtualDisk -ErrorAction SilentlyContinue | ForEach-Object {
    [pscustomobject]@{
        volume_name = $_.FriendlyName; volume_kind = 'virtual-disk'
        size_bytes  = [int64]$_.Size;  redundancy   = $_.ResiliencySettingName
    }
})

[pscustomobject]@{
    filesystems = $volumes
    storage_topology = @{ groups = $pools; volumes = $vdisk; disks = $disks }
} | ConvertTo-Json -Depth 6 -Compress
register: win_storage
changed_when: false

- name: "Windows | parser le JSON renvoyé"
  ansible.builtin.set_fact:
    win_storage_obj: "{{ (win_storage.output | join('')) | from_json }}"
  when: inventory_hostname in groups['windows']

```

5. Windows — pagefile + résumé ACL non sensible

```

- name: "Windows | pagefile + ACL summary (non sensible)"
  when: inventory_hostname in groups['windows']
  ansible.windows.win_powershell:
    script: |
      $pf = Get-CimInstance Win32_PageFileUsage | ForEach-Object {
        [pscustomobject]@{
            name          = $_.Name
            kind          = 'windows-pagefile'
            size_total_bytes = [int64]$_ .AllocatedBaseSize * 1MB
            size_used_bytes  = [int64]$_ .CurrentUsage * 1MB
        }
      }

      # Résumé ACL NON SENSIBLE : pas de SID nominatifs, juste comptes/principaux intégrés
      $acl = Get-Acl 'C:\'
      $aclSummary = [pscustomobject]@{
          owner_builtin      = ($acl.Owner -match 'BUILTIN|NT AUTHORITY')
          inheritance_enabled = (-not $acl.AreAccessRulesProtected)
          ace_count          = $acl.Access.Count
          has_system         = [bool]($acl.Access | Where-Object IdentityReference -match 'NT
AUTHORITY\SYSTEM')
          has_administrators = [bool]($acl.Access | Where-Object IdentityReference -match

```



```
'BUILTIN\\Administrators')
}
[pscustomobject]@{ paging = @($pf); system_volume_acl_summary = $aclSummary } |
    ConvertTo-Json -Depth 5 -Compress
register: win_paging_acl
changed_when: false
```

6. Agrégats de contenu bornés — du depth-1 (Unix) + Get-ChildItem (Windows)

```
- name: "Unix | du profondeur 1 borné (agrégats non sensibles)"
when: ansible_facts['system'] in ['Linux', 'AIX']
ansible.builtin.command:
    cmd: "du -x -b --max-depth=1 {{ item }}"    # -x : ne traverse pas les autres FS
loop: "{{ target_mounts | default(['/var', '/data']) }}"
register: unix_du
changed_when: false
failed_when: false
timeout: 120    # garde-fou : borne le temps de balayage

- name: "Windows | Get-ChildItem agrégé profondeur 1 (tailles seulement)"
when: inventory_hostname in groups['windows']
ansible.windows.win_powershell:
    script: |
        $root = 'D:\'
        Get-ChildItem -LiteralPath $root -Directory -Depth 0 -ErrorAction SilentlyContinue |
            ForEach-Object {
                $sum = (Get-ChildItem -LiteralPath $_.FullName -Recurse -File -ErrorAction
SilentlyContinue |
                    Measure-Object -Sum Length).Sum
                [pscustomobject]@{ path = $_.Name; size_bytes = [int64]$sum }
            } | ConvertTo-Json -Depth 3 -Compress
register: win_aggr
changed_when: false
```

7. Assemblage du document JSON agnostique via set_fact

```
- name: "Assembler le document canonique (un par hôte)"
ansible.builtin.set_fact:
    host_storage_doc:
        host_id: "{{ ansible_facts['machine_id'] | default(inventory_hostname) }}"
        hostname: "{{ ansible_facts['fqdn'] | default(inventory_hostname) }}"
        gsma_code: "{{ gsma_code | default(omit) }}"
        host_role_env: "{{ host_role_env | default('unknown') }}"
platform: >-
```

```

    {{ 'windows' if inventory_hostname in groups['windows']
      else ('aix' if ansible_facts['system'] == 'AIX' else 'linux') }}
os_family: "{{{ ansible_facts['os_family'] }}}"
os_distribution: "{{{ ansible_facts['distribution'] | default(ansible_os_name) }}}"
os_version: "{{{ ansible_facts['distribution_version'] |
default(ansible_distribution_version) }}}"
architecture: "{{{ ansible_facts['architecture'] | default(ansible_architecture) }}}"
snapshot_id: "{{{ ansible_date_time.iso8601_basic_short }}-{{{ inventory_hostname }}}"
scan_timestamp: "{{{ ansible_date_time.iso8601 }}}"
collection_method: "{{{ 'ansible-winrm' if inventory_hostname in groups['windows'] else
'ansible-ssh' }}}"
scan_confidence: "{{{ scan_confidence | default('high') }}}"
# Branches fusionnées : seule celle de l'OS courant est non vide
filesystems: "{{{ (win_storage_obj.filesystems if inventory_hostname in groups['windows']
else fs_unix) | default([]) }}}"
storage_topology: "{{{ (win_storage_obj.storage_topology if inventory_hostname in
groups['windows'] else topo_unix) | default({'groups': [], 'volumes': [], 'disks': []}) }}}"
paging: "{{{ paging_list | default([]) }}}"

- name: "Écrire le document JSON (artefact pour le pipeline LangGraph)"
  ansible.builtin.copy:
    content: "{{{ host_storage_doc | to_nice_json }}}"
    dest: "/var/lib/storage-scan/{{{ inventory_hostname }}}.json"
  delegate_to: localhost
  changed_when: true

```